

¿Puede el autojuego recordar su pasado?

Diseño experimental y resultado negativo en aprendizaje por refuerzo

Pedro Pérez Blanco

Basado en moanv2/rlgym (MIT) — el bot original no es obra mía

Informe técnico · 8 de septiembre de 2026

H1 INCONCLUYENTE · H2 NO-GO · Hipótesis sobre shaping FALSADA

Índice

Índice.....	2
1. Resumen ejecutivo.....	3
Resultados en una tabla.....	3
2. Pregunta de investigación.....	4
2.1 Contexto: aprendizaje por refuerzo y autojuego.....	4
3. Qué existía antes: la base heredada	5
4. Qué construí.....	6
4.1 El adaptador de rival congelado	6
4.2 El pool de rivales	6
4.3 Ejecución, evaluación e integridad	6
5. Diseño y resultado de H1	8
6. Diagnóstico posterior	9
7. H2 y la puerta GO/NO-GO.....	10
7.1 Un error de diseño propio	10
8. Diagnóstico de recompensas	11
9. La hipótesis sobre el shaping, y su falsación.....	12
10. Qué sabemos al cerrar	13
Confirmado	13
Falsado	13
No resuelto	13
11. Things I got wrong.....	14
12. Limitaciones	15
13. Reproducibilidad	16
13.1 Volumen del trabajo	16
14. Conclusiones	17
15. Referencias y atribución	18

1. Resumen ejecutivo

Este informe documenta un experimento de aprendizaje por refuerzo sobre Rocket League 1v1 y, sobre todo, el método con el que se midió. La pregunta era si entrenar contra un conjunto de versiones anteriores de uno mismo —un opponent pool— produce un agente mejor que entrenar contra una única copia congelada, con el mismo presupuesto de aprendizaje.

La respuesta honesta es que no se pudo demostrar. El experimento principal quedó inconcluyente; el intento de rehacerlo mejor se detuvo en su propia puerta de calidad antes de gastar el cómputo; y al investigar por qué apareció un problema más básico —entrenar más deterioraba el juego— para el que formulé una explicación que los datos terminaron falsando.

Lo que sí queda

Un sistema experimental con protocolos congelados por SHA256 antes de ejecutar, criterios de decisión escritos de antemano y respetados cuando el resultado incomodó, evaluación reproducible y trazabilidad por hashes. No hay un bot mejorado: hay un experimento que sabe decir que no.

Resultados en una tabla

Fase	Resultado	Detalle
H1	INCONCLUYENTE	A 5,00 % · B 4,17 % · B-A -0,83 pp · IC95 [-3,89, +2,22]
H2	NO-GO	G2 0,0310 < 0,0465 · G3 0,0135 < 0,0148 · no se entrenó
Recompensas	Ninguna prometedora	D_A 61,5 % · D_B 22,5 % · D_C 16,0 %
R1 / R2	FALSADA	Reducir el shaping bajó de 61,5 % a 2,0 %
Causa	NO IDENTIFICADA	Se sabe qué explicaciones no bastan

2. Pregunta de investigación

H1

Un agente entrenado contra una mezcla de instantáneas congeladas de sí mismo alcanza una tasa de victoria mayor, frente a rivales reservados, que uno entrenado contra una única copia congelada de su yo reciente, con idéntico presupuesto de muestras de aprendizaje.

Era razonable preguntarlo por tres motivos. La técnica está bien establecida a gran escala pero rara vez se mide a escala pequeña. El coste extra es casi nulo: guardar instantáneas es barato. Y la condición «con el mismo presupuesto» hace la comparación justa y falsable: si el pool ayuda, tiene que ayudar sin gastar más.

2.1 Contexto: aprendizaje por refuerzo y autojuego

El aprendizaje por refuerzo entrena a un agente sin decirle qué hacer: actúa, recibe una recompensa numérica y ajusta su comportamiento para acumular más. No hay ejemplos correctos que copiar, solo consecuencias.

El autojuego resuelve el problema de contra quién entrenar: el agente juega contra sí mismo, así que la dificultad se mantiene siempre al filo de lo que puede manejar. Pero tiene un fallo conocido: si entrenas siempre contra tu yo inmediatamente anterior puedes caer en ciclos, aprendiendo a batir la estrategia de ayer y olvidando cómo defenderte de la de anteayer. La solución clásica, el fictitious self-play, guarda un conjunto de versiones antiguas y entrena contra una mezcla de ellas.

La idea: entrenar contra tu propio pasado

En vez de una única copia congelada, un pool de versiones históricas

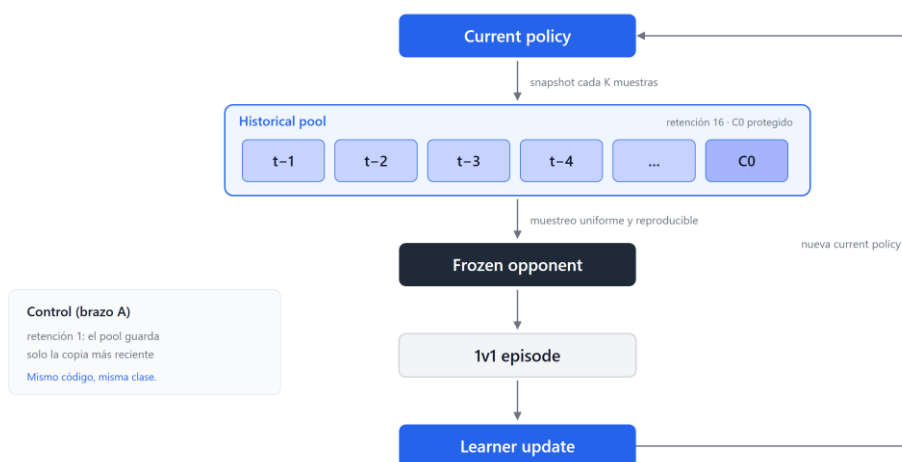


Figura 1. La idea. Cada K muestras se guarda una instantánea de la política. En cada episodio se muestrea un rival del pool. El control usa la misma clase con retención 1.

La Figura 1 resume el mecanismo. Los dos brazos del experimento usan el mismo código: lo único que cambia es cuántas instantáneas se conservan.

3. Qué existía antes: la base heredada

El proyecto parte de moanv2/rlgym, un trabajo final de la asignatura de Reinforcement Learning en IE School of Science and Technology, publicado con licencia MIT. Incluye el entorno 1v1, el entrenamiento con PPO, un registro de recompensas configurables y un arnés de torneo.

Atribución

El bot de Rocket League, su entorno de entrenamiento y su arnés de torneo son obra de sus autores originales. Este trabajo es una capa experimental construida encima. Ningún archivo heredado fue modificado, lo que se verifica con git status: 0 modificados.

Componente heredado	Qué aporta
PPO (rlgym-ppo)	recoge experiencia y mejora la política en pasos controlados
rlgym_sim + RocketSim	física de Rocket League sin gráficos, miles de pasos por segundo
Políticas	tres capas de 512; 89 números de entrada, 90 acciones discretas
Recompensas	registro por YAML, envuelto en ZeroSumReward
Arnés de torneo	enfrenta dos checkpoints y cuenta goles

4. Qué construí

Lo que no existía era nada para hacer un experimento controlado. Esa es la aportación propia: 5.401 líneas en 24 archivos, sin tocar ni uno heredado.

Arquitectura: qué construí encima de qué

En gris el proyecto heredado - en azul la capa experimental propia - en violeta los resultados



Fuente: docs/CONTRIBUTIONS.md

Figura 2. Arquitectura por autoría. En gris el proyecto heredado, en azul la capa experimental propia, en violeta los resultados.

La Figura 2 separa visualmente ambas cosas. A continuación, las piezas que más condicionan la validez del experimento.

4.1 El adaptador de rival congelado

Es la pieza crítica. PPO espera un entorno con un agente; Rocket League 1v1 tiene dos. Si se le entregan los dos, PPO trata al rival como un segundo aprendiz y su experiencia entra en las actualizaciones: estarías entrenando con datos del oponente y midiendo algo distinto de lo que crees. El adaptador expone un solo agente; el rival actúa dentro de `step()`, congelado y sin gradiente.

4.2 El pool de rivales

Instantáneas cada K muestras, manifiesto escrito de forma atómica, un SHA256 por instantánea, muestreo reproducible a partir de la semilla y retención con protección del punto de partida. Los dos brazos usan la misma clase: si cada uno usara código distinto, cualquier diferencia podría venir del código.

4.3 Ejecución, evaluación e integridad

- Una corrida por proceso, sin excepción, con detección de estancamiento y de procesos huérfanos.
- Marcador explícito COMPLETA/INCOMPLETA en disco; una corrida incompleta se rehace entera, nunca se continúa a medias.

- Evaluación en JSONL con identificador único por partida y los hashes de las dos políticas que jugaron; reanudar no repite partidas.
- Tabla de integridad ciega: comprueba que las corridas son comparables sin mirar ningún resultado de juego, y recupera cada checkpoint en un proceso nuevo.
- Protocolos congelados por SHA256: los scripts se niegan a ejecutarse si el archivo cambió. Es lo que impide ajustar el criterio después de ver los resultados.

5. Diseño y resultado de H1

Decisión	Valor	Por qué
C0	32.000 muestras, hash fijado	ambos brazos parten de lo mismo
Recompensa	stage_1_basics heredada	no inventar pesos nuevos
K	40.000	con 8.000 las instantáneas salían casi clones
Retención	$A = 1 \cdot B = 16$, C0 protegido	es la única diferencia entre brazos
Presupuesto	500.000 muestras por corrida	mínimo con 12 rivales distinguibles
Semillas	20260907/08/09, emparejadas	fijadas antes de ver nada
Evaluación	720 partidas, rivales reservados	no usados en decisiones previas

El criterio de decisión se congeló con SHA256 8ae06382... antes de entrenar: H1 se declara a favor solo si se cumplen cuatro condiciones a la vez, entre ellas que el intervalo de confianza excluya el cero y que la magnitud supere el ruido entre semillas.

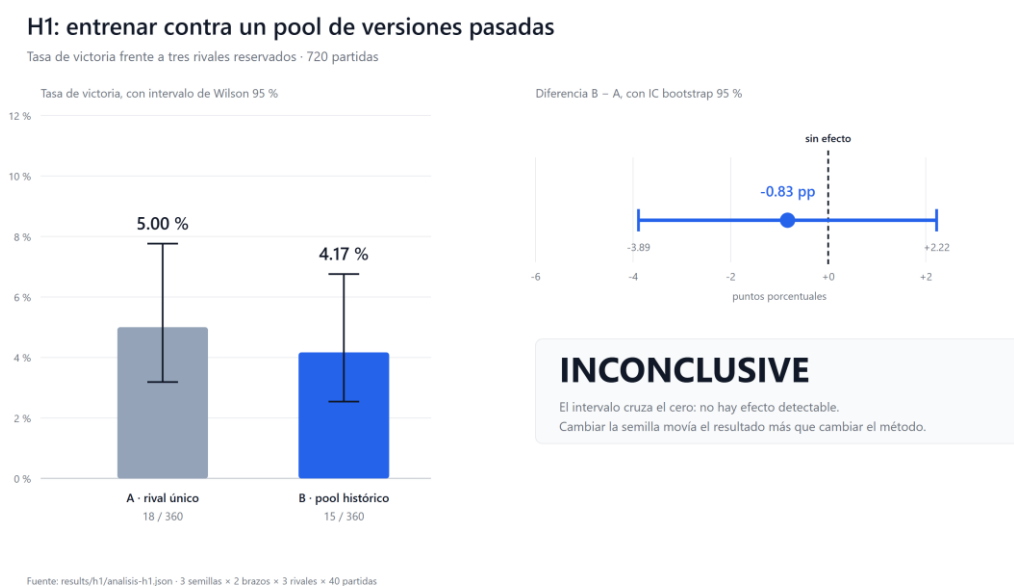


Figura 3. Resultado de H1. Tasas con intervalo de Wilson y diferencia B-A con IC bootstrap. El intervalo de la diferencia cruza el cero.

La Figura 3 muestra el resultado: A 5,00 %, B 4,17 %, diferencia **-0,83 puntos porcentuales** con intervalo **[-3,89, +2,22]**.

Veredicto: INCONCLUYENTE

Esto no significa que A sea mejor que B. El intervalo contiene el cero con holgura: los datos son compatibles con que el pool ayude, con que perjudique y con que no haga nada. La variación entre semillas del mismo brazo (2,5 pp en A) supera a la diferencia entre brazos (0,83 pp): cambiar la semilla movía el resultado más que cambiar el método.

6. Diagnóstico posterior

Tres medidas sobre los checkpoints que H1 dejó en disco explican el resultado mejor que cualquier excusa.

Hallazgo	Medida	Consecuencia
Efecto suelo	ambos brazos ~5 %	rivales entrenados unas 4.000 veces más
Deriva mínima	1,55 % desde C0	la política apenas se movió en 504.000 muestras
Pool clónico	mediana 0,0074	las 13 instantáneas, dentro del 1,5 % unas de otras

El punto incómodo

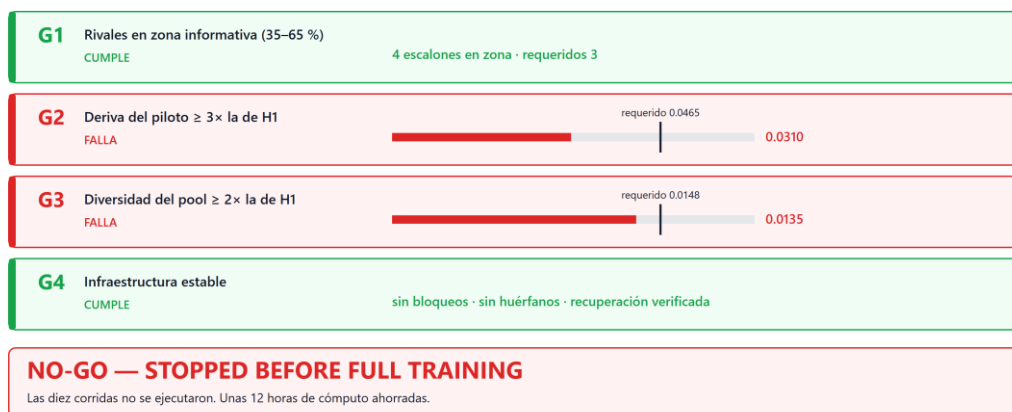
El tratamiento apenas se diferenciaba del control. Entrenar contra 13 copias casi idénticas de uno mismo no es muy distinto de entrenar contra una sola. H1 no solo carecía de potencia estadística: la intervención que medía era muy débil. Además, la regla de retención nunca llegó a activarse, porque salieron 13 instantáneas frente a un tope de 16.

7. H2 y la puerta GO/NO-GO

H2 se diseñó para corregir exactamente eso: rivales calibrados mediante un linaje neutral de sparring, presupuesto de 2,5 millones, K de 100.000 para que el pool cubriera más historia, cinco semillas y un bootstrap jerárquico que respeta la corrida como unidad experimental. Y algo que H1 no tenía: una puerta con umbrales escritos antes, que había que superar antes de gastar el cómputo.

La puerta de H2: umbrales escritos antes de gastar el cómputo

Se evalúa antes de entrenar. Si falla alguno, el experimento no se ejecuta.



Fuente: results/h2/puerta.json

Figura 4. La puerta de H2. Cuatro criterios con su umbral y su valor medido. Dos fallan.

Como recoge la Figura 4, G1 y G4 se cumplieron —el linaje neutral sí produjo rivales en zona informativa— pero G2 y G3 fallaron. Las diez corridas no se ejecutaron.

7.1 Un error de diseño propio

G2 falló porque yo había extrapolado la deriva en línea recta. Crece como raíz cuadrada del presupuesto, y el linaje neutral lo confirmó punto por punto: 0,0156 a 504.000 muestras, 0,0223 a 1 millón, 0,0305 a 2 y 0,0403 a 4. Alcanzar el umbral habría exigido unos 6 millones por corrida.

Por qué respetar la puerta importa

Un NO-GO no es un fallo de software: es el sistema funcionando. La tentación era bajar G2 de $3 \times$ a $2 \times$, que era justo lo medido. Habría sido elegir el umbral después de ver el dato, es decir, exactamente lo que la puerta existía para impedir. Lo que se ganó: no gastar 12 horas de cómputo en un experimento cuya premisa ya no se sostenía.

8. Diagnóstico de recompensas

Durante la calibración apareció algo que nadie había pedido mirar: el agente piloto, con cinco veces el presupuesto de H1, perdía contra su propio punto de partida. Eso invalidaba la premisa de comparar métodos por tasa de victoria, así que se investigó con tres configuraciones fijadas de antemano.

Tres recompensas, ninguna con mejora sostenida

Tasa de victoria contra C0 · 200 partidas por punto



Ninguna mejora de forma sostenida al aumentar el presupuesto.

Si entrenar más empeora, comparar dos métodos por tasa de victoria compara dos formas de empeorar.

Fuente: results/reward-diagnostic/resumenes/

Figura 5. Diagnóstico de recompensas. Tasa de victoria contra C0 con 200 partidas por punto. La línea discontinua marca el empate con el punto de partida.

La Figura 5 muestra las tres trayectorias. D_A, con la recompensa heredada, es la única que supera el 50 %, pero cae del 72,0 % al 61,5 %: diez puntos y medio, 2,2 errores típicos. Falló el criterio de deterioro por 0,5 puntos porcentuales, y se aplicó como estaba escrito.

Sobre ZeroSumReward: su efecto cambia de signo entre presupuestos, así que con una sola semilla no puede aislarse, y no es el factor dominante.

9. La hipótesis sobre el shaping, y su falsación

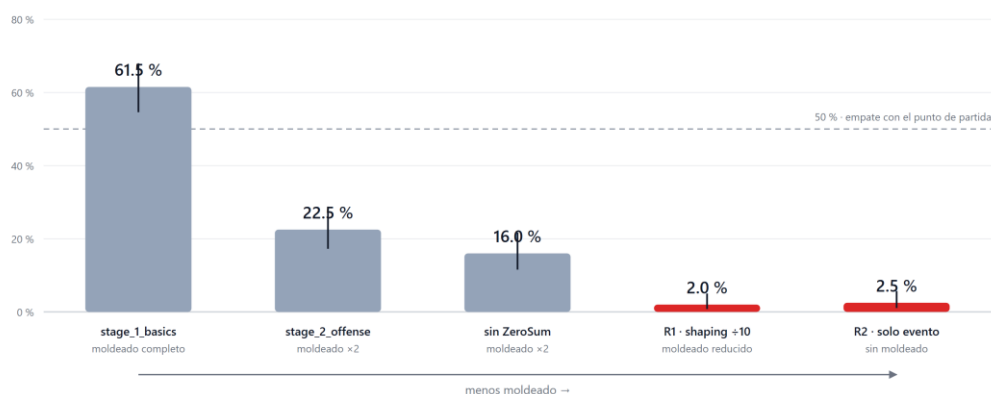
De ahí salió una explicación con números detrás. Las recompensas mezclan términos moldeados, que se cobran en cada paso, y un término de evento, que premia el gol una sola vez. La aritmética: 0,30 por paso durante episodios de hasta 300 pasos, frente a 8,0 una vez. El moldeado puede dominar al objetivo real en un orden de magnitud.

La predicción falsable

Si el moldeado ahoga al objetivo, reducirlo debería mejorar el aprendizaje. Se probaron dos configuraciones derivadas literalmente de la heredada: R1 con los términos densos divididos por diez, y R2 solo con el término de evento.

Mi hipótesis: el moldeado ahogaba al objetivo. Predicción: reducirlo ayudaría.

Tasa de victoria contra C0 tras 500.000 muestras · 200 partidas por barra · ordenado de más a menos moldeado



Reducir el moldeado lo empeoró: de 61,5 % a 2,0 %,

La predicción quedó falsada. La causa del deterioro sigue sin identificarse.

Fuente: results/reward-diagnostic/resumenes/ y results/reward-final-check/resumenes/ - IC de Wilson 95 %

Figura 6. La falsación. Tasa contra C0 tras 500.000 muestras, ordenada de más a menos moldeado. La predicción apuntaba en sentido contrario.

La Figura 6 muestra lo contrario de lo predicho: cuanto menos moldeado, **peor**. Dividirlo por diez hundió el rendimiento del 61,5 % al 2,0 %.

No fue un fallo de estabilidad: ambas entrenaron sin error, con parámetros finitos y la entropía alta. Mi aritmética era correcta pero irrelevante: confundí «qué término domina la suma de la recompensa» con «qué término aporta señal aprovechable». Sin moldeado, los goles son tan raros que la inmensa mayoría de los pasos no informan de nada.

No propongo una causa alternativa

No la tengo. La causa del deterioro sigue sin identificarse.

10. Qué sabemos al cerrar

Confirmado

- H1 quedó inconcluyente: $B-A = -0,83$ pp, IC95 $[-3,89, +2,22]$.
- La variación entre semillas supera a la diferencia entre brazos.
- Tras 504.000 muestras la política se aleja solo un 1,55 % del inicio.
- Las instantáneas del pool eran casi idénticas: mediana 0,0074.
- La deriva crece como raíz cuadrada, verificado en cinco puntos.
- La retención funciona: 41 instantáneas creadas, 16 conservadas, 25 expulsadas.
- En las cinco recompensas probadas, entrenar más degrada la tasa de victoria.
- Reducir o eliminar el moldeado hunde el rendimiento: 61,5 % $\rightarrow$ 2,0 %.

Falsado

- Que el exceso de reward shaping causara el deterioro.
- Que DefaultReward fuese «solo goles»: penaliza la velocidad angular.
- Que la varianza explicada sirviera para validar el crítico aquí.
- Que los errores de cierre de sockets fueran inofensivos.
- Que todos los agentes entrenados perdieran contra C0: con 200 partidas, la recompensa heredada gana el 61,5 %.

No resuelto

La pregunta viva

Por qué entrenar más deteriora el juego frente al punto de partida. Es el hallazgo central y no tiene explicación identificada. Se sabe qué explicaciones no bastan: no es exceso de moldeado, no es la envoltura ZeroSum, no es inestabilidad numérica y no es sobreajuste al rival, porque el rival de entrenamiento y el de evaluación son el mismo.

11. Things I got wrong

Las rectificaciones forman parte del trabajo. Cada una con lo mismo: qué supuse, cómo lo comprobé, qué encontré.

Suposición	Cómo se comprobó	Qué encontré
DefaultReward premia solo goles	leyendo el código en vez de repetirlo	penaliza la velocidad angular; no codifica la tarea
La varianza explicada valida el crítico	mirando de dónde salen los retornos	es circular: $ret = val + adv$
La razón valor/retorno calibra	midiéndola bajo ZeroSumReward	con retorno medio ≈ 0 se dispara a 48,3
Todos pierden contra el inicio	repetiendo con 200 partidas en vez de 40	la heredada gana el 61,5 %; mi afirmación era demasiado fuerte
El exceso de shaping causa el deterioro	dos configuraciones con criterio congelado	lo contrario: 61,5 % $\rightarrow$ 2,0 %
Los WinError 10038 son inofensivos	cuando la corrida siguiente se colgó	lo son para la que termina, no para la siguiente del mismo proceso

12. Limitaciones

- Presupuesto muy pequeño: 504.000 muestras por corrida, frente a los miles de millones del proyecto original. Escala de prueba de concepto.
- Efecto suelo en H1: rivales fuera de alcance, tasas del 5 %.
- Tres semillas en H1 y una sola por configuración en los diagnósticos: no permite separar el efecto del método del de la semilla.
- La retención nunca se activó en H1: figuraba en el diseño sin ejercer efecto.
- El bootstrap de H1 remuestreaba partidas como si fueran independientes; la unidad real es la corrida. No cambió el veredicto, pero el intervalo salía más estrecho de lo debido.
- Máquina única, CPU, sin GPU: entre 1.100 y 1.350 muestras por segundo.

13. Reproducibilidad

Nivel	Qué incluye
Reproducible directamente	los análisis, desde los JSONL publicados: veredicto de H1, tabla de integridad y puerta de H2
Requiere artefactos locales	los entrenamientos y evaluaciones: necesitan las mallas de colisión y el punto de partida
No redistribuible	mallas de colisión y checkpoints de terceros
Costoso de repetir	las seis corridas de H1 (~1 h) y el linaje neutral de 4 M (~50 min)

El entrenamiento con dos trabajadores no es reproducible bit a bit: el orden de llegada de la experiencia varía y los hashes de una repetición no coincidirán. Es una limitación real y está declarada.

13.1 Volumen del trabajo

Magnitud	Valor
Entrenamientos con informe	15
Muestras procesadas	12.112.000
Cómputo registrado	3,00 h
Partidas evaluadas	3.960
Protocolos congelados por SHA256	4
Código propio	5.401 líneas en 24 archivos
Archivos heredados modificados	0

Las 3.960 partidas son una magnitud de ingeniería, no una muestra estadística: proceden de protocolos distintos y no deben agregarse para calcular ninguna tasa conjunta.

14. Conclusiones

El resultado científico

Con este entorno, este presupuesto y esta configuración no se pudo demostrar una ventaja del opponent pool. La investigación posterior reveló limitaciones más fundamentales en el proceso de aprendizaje, y una hipótesis explicativa formulada después fue falsada experimentalmente.

La conclusión no es «el opponent pool no funciona». La diferencia entre ambas frases no es cosmética: la primera afirma algo sobre el mundo que estos datos no sostienen; la segunda afirma algo sobre lo que este experimento podía y no podía detectar, que es exactamente lo que se midió.

No construí un bot mejor. No demostré que el opponent pool ayude. Y la explicación que propuse para el problema que encontré resultó ser falsa. Lo que sí construí es un sistema que podía decir que no: que dio «inconcluyente» cuando los datos eran ambiguos, que se detuvo en su propia puerta por 0,0155 de deriva en vez de bajar el umbral, que descartó una configuración por 0,5 puntos porcentuales porque el criterio estaba fijado de antemano, y que tumbó la hipótesis de quien lo construyó.

Los pesos de una política de Rocket League no le sirven a nadie. Un procedimiento que impide engañarse a uno mismo, sí.

15. Referencias y atribución

Elemento	Detalle
Proyecto original	moanv2/rlgym — https://github.com/moanv2/rlgym
Commit base	0c6965acf3d0405227282a80b1881dbfde3ef56b
Licencia	MIT — © 2026 Diego Alfaro Gomez
Origen	Final Project for Reinforcement Learning, IE School of Science and Technology
Aportación propia	infraestructura experimental, opponent pool, reproducibilidad, evaluación, análisis y metodología
Archivos heredados modificados	ninguno

Atribución

El bot de Rocket League y su entorno de entrenamiento son obra de sus autores originales. Este trabajo no los sustituye ni reclama su autoría. No se afirma haber mejorado el bot original ni haber superado ninguna línea base.

Documentación completa del proyecto: PROJECT_STORY.md (historia con figuras), FINAL_REPORT.md (informe técnico), CONTRIBUTIONS.md (autoría línea a línea) y REPRODUCE.md (guía de reproducción).